

Workshop: Unüberwachtes Lernen mit Python & TensorFlow (Autoencoder)

Überblick

In diesem Workshop trainierst du ein **Autoencoder**-Modell (unsupervised) mit TensorFlow/Keras, um Muster in einem Datensatz zu lernen.

Du wirst: 1. einen Datensatz einlesen, 2. die Daten vorbereiten, 3. ein Autoencoder-Modell bauen und trainieren, 4. das Ergebnis evaluieren (Rekonstruktionsfehler) und 5. die Resultate visualisieren.

Der Code ist so vorbereitet, dass du **Hyperparameter gezielt anpassen** kannst, um Effekte auf Training, Fehlerverteilung und Anomalie-Erkennung zu untersuchen.

Lernziele

- Unterschied zwischen überwachten und unüberwachten Lernaufgaben verstehen (hier: Rekonstruktion statt Klassifikation).
 - Datenvorbereitung (Train/Test-Split, Skalierung).
 - Architektur eines Autoencoders in Keras aufbauen und trainieren.
 - Evaluation über **Rekonstruktionsfehler** und **Schwellwert** (Quantil) zur Anomalie-Detektion.
 - Visualisierung: Loss-Verlauf, Fehler-Histogramm, Latent-Space-Projektion.
-

Voraussetzungen

- **Python 3.9+** (empfohlen)
- Folgende Pakete:

```
pip install tensorflow scikit-learn pandas matplotlib numpy
```

- Optional: eigener CSV-Datensatz mit numerischen Spalten.
-

Daten

Standardmäßig nutzt das Skript den eingebauten **Iris**-Datensatz (numerisch, 4 Features) als *Fallback* und speichert ihn als `iris_fallback.csv`.

Du kannst alternativ **einen eigenen CSV-Datensatz** nutzen (z. B. `data/workshop_data.csv`). Nicht-numerische Spalten werden automatisch verworfen.

Tipp: Je mehr Features und Zeilen, desto spannender wird der Latent Space.

Aufgabe

1. **Starte das Skript** (siehe „Ausführen“ unten) und beobachte Trainingslogs sowie Plots.
 2. **Passe Hyperparameter** im CONFIG-Block an (siehe Liste unten). Notiere, wie sich
 - Trainings-/Validierungsverlust,
 - die Verteilung der Rekonstruktionsfehler und
 - die Anzahl erkannter „Anomalien“ (über dem Schwellwert) verändern.
 3. **Variiere** insbesondere:
 - LATENT_DIM (Größe des Engpasses),
 - HIDDEN_DIMS (Tiefe/Breite),
 - LEARNING_RATE, EPOCHS, BATCH_SIZE,
 - Regularisierung (L2, DROPOUT),
 - THRESHOLD_QUANTILE (z. B. 0.90, 0.95, 0.99),
 - NOISE_FACTOR (für einen Denoising-Autoencoder).
 4. **Dokumentiere** kurz deine besten Einstellungen und beschreibe die beobachteten Effekte.
 5. **Bonus (optional)**:
 - Füge BatchNormalization-Layer ein und vergleiche.
 - Visualisiere den Latent Space zusätzlich mit **PCA** auf 2D, falls LATENT_DIM > 2.
 - Wende **KMeans** im Latent Space an und bewerte die Trennbarkeit der Cluster (z. B. mit Silhouette Score).
-

Hyperparameter (im Code oben gesammelt)

- TEST_SIZE: Anteil Testdaten (z. B. 0.2)
- LATENT_DIM: Dimensionalität des Engpasses (z. B. 2–8)
- HIDDEN_DIMS: Liste der Dense-Layer-Größen im Encoder (Decoder spiegelt das)
- ACTIVATION: Aktivierungsfunktion (z. B. 'relu', 'elu', 'selu')
- DROPOUT: Dropout-Rate (0.0–0.5)
- L2: L2-Regularisierung (1e-6–1e-3)
- LEARNING_RATE: Lernrate des Optimizers (1e-4–1e-2)
- BATCH_SIZE: Minibatch-Größe (z. B. 32–256)
- EPOCHS: Trainings-Epochen (z. B. 50–300)
- PATIENCE: Early-Stopping-Patience (z. B. 15)
- VALIDATION_SPLIT: Anteil Validierungsdaten aus Training (z. B. 0.2)
- THRESHOLD_QUANTILE: Quantilschwelle für Anomalie-Flag (z. B. 0.95)

- NOISE_FACTOR: Rauschanteil für Denoising-AE (z. B. 0.0–0.3)
 - RANDOM_SEED: Reproduzierbarkeit
-

Ausführen

Variante A: Standard (Iris-Fallback)

```
python workshop_autoencoder.py
```

Variante B: Eigene CSV nutzen

```
python workshop_autoencoder.py --data data/workshop_data.csv
```

Hinweis: Nicht-numerische Spalten werden verworfen, fehlende Werte werden zeilenweise entfernt.

Starter-Code (Editiere den CONFIG-Block!)

Speicher diese Datei als `workshop_autoencoder.py` im selben Ordner und führe sie aus.

```
import os
import argparse
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers, callbacks

# =====
# HYPERPARAMETER-KONFIG
# =====
CONFIG = {
    # Daten & Split
    "TEST_SIZE": 0.2,
    "VALIDATION_SPLIT": 0.2,
    "RANDOM_SEED": 42,
```

```

# Modellarchitektur
"LATENT_DIM": 2,                # << Experimentiere: 2, 3, 4, 8 ...
"HIDDEN_DIMS": [32, 16],       # << Experimentiere: [64,32], [128,64,32], ...
"ACTIVATION": "relu",
"DROPOUT": 0.0,                # << 0.0 bis 0.5
"L2": 1e-5,                    # << 0 bis 1e-3

# Training
"LEARNING_RATE": 1e-3,         # << 1e-4 bis 1e-2
"BATCH_SIZE": 64,              # << 32, 64, 128, 256 ...
"EPOCHS": 150,
"PATIENCE": 15,

# Anomalie-Detektion
"THRESHOLD_QUANTILE": 0.95,     # << 0.9-0.99

# Denoising
"NOISE_FACTOR": 0.0,           # << 0.0-0.3 (z.B. 0.1)
}

def set_seeds(seed: int):
    np.random.seed(seed)
    tf.random.set_seed(seed)

def load_dataset(path: str | None) -> pd.DataFrame:
    """
    Lädt einen DataFrame. Falls path nicht existiert, wird Iris als Fallback verwendet.
    Nur numerische Spalten werden behalten; Zeilen mit NaNs werden entfernt.
    """
    if path and os.path.exists(path):
        print(f"[INFO] Lade Datensatz aus: {path}")
        df = pd.read_csv(path)
    else:
        print("[INFO] Kein/ungültiger Pfad. Verwende Iris-Fallback und speichere iris_fallback.csv")
        iris = load_iris()
        df = pd.DataFrame(iris.data, columns=[c.replace(' (cm)', '') for c in iris.feature_names])
        df.to_csv("iris_fallback.csv", index=False)

    df_num = df.select_dtypes(include=[np.number]).copy()
    before = len(df_num)
    df_num.dropna(axis=0, how="any", inplace=True)
    after = len(df_num)
    if after < before:
        print(f"[WARN] {before - after} Zeilen mit fehlenden Werten entfernt.")

```

```

print(f"[INFO] Datenform (numerisch, ohne NaNs): {df_num.shape}")
return df_num

def standardize(train_df: pd.DataFrame, test_df: pd.DataFrame) -> tuple[np.ndarray, np.ndarray]:
    scaler = StandardScaler()
    X_train = scaler.fit_transform(train_df.values)
    X_test = scaler.transform(test_df.values)
    return X_train, X_test, scaler

def build_autoencoder(input_dim: int, cfg: dict) -> keras.Model:
    l2 = regularizers.l2(cfg["L2"]) if cfg["L2"] > 0 else None

    inputs = keras.Input(shape=(input_dim,), name="inputs")
    x = inputs
    for units in cfg["HIDDEN_DIMS"]:
        x = layers.Dense(units, activation=cfg["ACTIVATION"], kernel_regularizer=l2)(x)
        if cfg["DROPOUT"] and cfg["DROPOUT"] > 0:
            x = layers.Dropout(cfg["DROPOUT"])(x)

    latent = layers.Dense(cfg["LATENT_DIM"], activation=None, name="latent")(x)

    x = latent
    for units in reversed(cfg["HIDDEN_DIMS"]):
        x = layers.Dense(units, activation=cfg["ACTIVATION"])(x)
    outputs = layers.Dense(input_dim, activation=None, name="outputs")(x)

    model = keras.Model(inputs=inputs, outputs=outputs, name="autoencoder")
    opt = keras.optimizers.Adam(learning_rate=cfg["LEARNING_RATE"])
    model.compile(optimizer=opt, loss="mse")
    return model

def add_noise(X: np.ndarray, noise_factor: float, seed: int) -> np.ndarray:
    if noise_factor <= 0:
        return X
    rng = np.random.default_rng(seed)
    noise = noise_factor * rng.normal(loc=0.0, scale=1.0, size=X.shape)
    noisy = X + noise
    return noisy

def plot_training(history: keras.callbacks.History):
    plt.figure()
    plt.plot(history.history["loss"], label="train")

```

```

if "val_loss" in history.history:
    plt.plot(history.history["val_loss"], label="val")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training & Validation Loss")
plt.legend()
plt.tight_layout()
plt.show()

def reconstruction_errors(model: keras.Model, X: np.ndarray) -> np.ndarray:
    X_pred = model.predict(X, verbose=0)
    errors = np.mean(np.square(X - X_pred), axis=1)
    return errors

def plot_error_hist(errors_train: np.ndarray, errors_test: np.ndarray, threshold: float):
    plt.figure()
    plt.hist(errors_train, bins=30, alpha=0.6, label="Train")
    plt.hist(errors_test, bins=30, alpha=0.6, label="Test")
    plt.axvline(threshold, linestyle="--", label=f"Threshold={threshold:.4f}")
    plt.xlabel("Rekonstruktionsfehler (MSE)")
    plt.ylabel("Häufigkeit")
    plt.title("Fehlerverteilung")
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_latent_space(encoder: keras.Model, X: np.ndarray, errors: np.ndarray, cfg: dict):
    # encoder: Inputs -> latent layer
    # Baue einen Encoder-Teil aus dem Autoencoder:
    latent_layer = encoder.get_layer("latent")
    latent_model = keras.Model(inputs=encoder.input, outputs=latent_layer.output)

    Z = latent_model.predict(X, verbose=0)

    if Z.shape[1] >= 2:
        z2 = Z[:, :2]
    else:
        # PCA auf 2D projizieren, falls latent < 2
        pca = PCA(n_components=2, random_state=cfg["RANDOM_SEED"])
        z2 = pca.fit_transform(Z)

    plt.figure()
    plt.scatter(z2[:, 0], z2[:, 1], c=errors, s=20)

```

```

plt.xlabel("Latent 1 / PCA-1")
plt.ylabel("Latent 2 / PCA-2")
plt.title("Latent-Space (Farben = Rekonstruktionsfehler)")
plt.colorbar(label="MSE")
plt.tight_layout()
plt.show()

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", type=str, default=None, help="Pfad zu einer CSV-Datei (optional)")
    args = parser.parse_args()

    cfg = CONFIG.copy()
    set_seeds(cfg["RANDOM_SEED"])

    # 1) Daten laden
    df = load_dataset(args.data)

    # 2) Split & Skalierung
    train_df, test_df = train_test_split(df, test_size=cfg["TEST_SIZE"], random_state=cfg["RANDOM_SEED"])
    X_train, X_test, scaler = standardize(train_df, test_df)

    # 3) Denoising (optional)
    X_train_in = add_noise(X_train, cfg["NOISE_FACTOR"], seed=cfg["RANDOM_SEED"])

    # 4) Modell bauen
    model = build_autoencoder(input_dim=X_train.shape[1], cfg=cfg)
    model.summary()

    # 5) Trainieren
    es = callbacks.EarlyStopping(monitor="val_loss", patience=cfg["PATIENCE"], restore_best_weights=True)
    history = model.fit(
        X_train_in, X_train,
        validation_split=cfg["VALIDATION_SPLIT"],
        epochs=cfg["EPOCHS"],
        batch_size=cfg["BATCH_SIZE"],
        callbacks=[es],
        verbose=1
    )

    # 6) Training visualisieren
    plot_training(history)

    # 7) Rekonstruktionsfehler berechnen
    train_errors = reconstruction_errors(model, X_train)

```

```

test_errors = reconstruction_errors(model, X_test)

# Threshold aus Train-Fehlern ableiten
threshold = float(np.quantile(train_errors, cfg["THRESHOLD_QUANTILE"]))
test_anomalies = (test_errors > threshold)

print(f"""\
[RESULT]
Threshold (Quantil {cfg["THRESHOLD_QUANTILE"]:.2f}): {threshold:.6f}
Train-Fehler: mean={np.mean(train_errors):.6f}, std={np.std(train_errors):.6f}
Test-Fehler: mean={np.mean(test_errors):.6f}, std={np.std(test_errors):.6f}
Anteil Test-"Anomalien": {100.0 * np.mean(test_anomalies):.2f}% \
""")

# 8) Fehlerverteilung visualisieren
plot_error_hist(train_errors, test_errors, threshold)

# 9) Latent-Space visualisieren (Farbe = Fehler)
plot_latent_space(model, X_test, test_errors, cfg)

if __name__ == "__main__":
    main()

```

Reflexion (nach dem Experimentieren)

- Welche Hyperparameter hatten den größten Einfluss auf **Generalisation** (Gap train/val)?
- Wie verändert sich die **Fehlerverteilung** und der Anteil „Anomalien“, wenn du
 - LATENT_DIM erhöhst/verkleinerst?
 - Regularisierung (L2, DROPOUT) aktivierst?
 - ein **Denoising**-Setup nutzt (NOISE_FACTOR > 0)?
- Wie interpretierst du die **Latent-Space-Plots**?